

Manipal School of Information Sciences

Linux Operating System Theory Assignment

Hard Links, Symbolic Links and the sort Command

Link Experiments and Sorting Security Data

Student Name:	Ribhav Devprasad
Reg. No.:	26110690002
Date:	3 November 2026

Hard Links vs. Symbolic Links, and sort

Objective and setup

This experiment compares a hard link and a symbolic (soft) link to the same file, and then applies `sort` to security data. Every command was executed in a scratch directory and the output shown is the real output. Inode numbers and timestamps differ from machine to machine; the relationships between them (equal or different, increasing or unchanged) are what matter.

Part A: Create the links

```
$ mkdir link_lab && cd link_lab && pwd
/tmp/lab4/link_lab
```

```
$ cat > security.txt <<'EOF'
Linux File System Security Lab
Confidential Security Report
EOF
cat security.txt
Linux File System Security Lab
Confidential Security Report
```

```
$ ln security.txt security_hard.txt        # hard link
ln -s security.txt security_soft.txt     # symbolic link
ls -li
total 8
2801705 -rw-r--r-- 2 root root 60 Nov  4 04:09 security.txt
2801705 -rw-r--r-- 2 root root 60 Nov  4 04:09 security_hard.txt
2801706 lrwxrwxrwx 1 root root 12 Nov  4 04:09 security_soft.txt -> security.txt
```

Observed inode numbers

File	Inode number
security.txt	2801705
security_hard.txt	2801705
security_soft.txt	2801706

Difference observed. `security.txt` and `security_hard.txt` have the **same inode number**: they are two directory entries (names) for one file, so neither is the "original". `security_soft.txt` has a **different inode**: it is a separate, tiny file whose content is the pathname `security.txt`.

Part B: Examine file metadata

```
$ ls -l security.txt security_hard.txt security_soft.txt
-rw-r--r-- 2 root root 60 Nov  4 04:09 security.txt
```

```
-rw-r--r-- 2 root root 60 Nov  4 04:09 security_hard.txt
lrwxrwxrwx 1 root root 12 Nov  4 04:09 security_soft.txt -> security.txt
```

The first character of the permission field gives the file type: `-` for `security.txt` and `security_hard.txt` (regular files) and `l` for `security_soft.txt` (symbolic link).

Link counts (second column): the original and the hard link both show **2**, because the one inode now has two names. The symbolic link shows **1**: it is its own inode with a single name, and creating a symlink does not increase the target's link count.

```
$ stat security.txt security_hard.txt security_soft.txt
  File: security.txt
  Size: 60              Blocks: 8          IO Block: 4096   regular file
Device: 254,0   Inode: 2801705     Links: 2
Access: (0644/-rw-r--r--)  Uid: (    0/    root)   Gid: (    0/    root)
Access: 2026-11-04 04:09:34.200692130 +0000
Modify: 2026-11-04 04:09:34.199469586 +0000
Change: 2026-11-04 04:09:34.203743046 +0000
 Birth: 2026-11-04 04:09:34.193086667 +0000
  File: security_hard.txt
  Size: 60              Blocks: 8          IO Block: 4096   regular file
Device: 254,0   Inode: 2801705     Links: 2
Access: (0644/-rw-r--r--)  Uid: (    0/    root)   Gid: (    0/    root)
Access: 2026-11-04 04:09:34.200692130 +0000
Modify: 2026-11-04 04:09:34.199469586 +0000
Change: 2026-11-04 04:09:34.203743046 +0000
 Birth: 2026-11-04 04:09:34.193086667 +0000
  File: security_soft.txt -> security.txt
  Size: 12              Blocks: 0          IO Block: 4096   symbolic link
Device: 254,0   Inode: 2801706     Links: 1
Access: (0777/lrwxrwxrwx)  Uid: (    0/    root)   Gid: (    0/    root)
Access: 2026-11-04 04:09:34.206218593 +0000
Modify: 2026-11-04 04:09:34.203743046 +0000
Change: 2026-11-04 04:09:34.203743046 +0000
 Birth: 2026-11-04 04:09:34.203743046 +0000
```

Recorded information

Parameter	Original	Hard Link	Soft Link
Inode number	2801705	2801705	2801706
File type	regular file	regular file	symbolic link
Size	60 bytes	60 bytes	12 bytes
Link count	2	2	1
Permissions	-rw-r--r-- (644)	-rw-r--r-- (644)	lrwxrwxrwx (777)

File sizes. The original and hard link have the same size because they are the same data. The symbolic link's size is the **length of the pathname it stores** (`security.txt` is 12 characters), not the size of the target's content. Its permissions (`rwxrwxrwx`) are always shown open because access is decided by the target file's permissions.

Part C: Investigate readlink

```
$ readlink security_soft.txt
security.txt
```

```
$ readlink security_hard.txt; echo "exit status: $?"
exit status: 1
```

```
$ readlink -f security_soft.txt
/tmp/lab4/link_lab/security.txt
```

`readlink` on the soft link prints the stored target, `security.txt` (a relative path). On the hard link it prints **nothing** and exits with status 1, because a hard link is an ordinary directory entry, not a link object: there is no stored pathname to read. `readlink -f` follows every symbolic link and prints the full, absolute, canonical path of the final target, which shows where the symlink really leads.

Part D: Content modification

```
$ echo "Unauthorized login detected" >> security.txt
```

```
$ echo '--- security.txt ---'; cat security.txt; echo '--- security_hard.txt ---'; cat
security_hard.txt; echo '--- security_soft.txt ---'; cat security_soft.txt
--- security.txt ---
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
--- security_hard.txt ---
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
--- security_soft.txt ---
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
```

The new line is visible through all three names. The hard link shares the same inode and therefore the same data blocks. The symbolic link looks up `security.txt` by name each time it is opened and so reaches the same inode.

```
$ echo "Modified via hard link" >> security_hard.txt
echo '--- security.txt ---'; cat security.txt; echo '--- security_soft.txt ---'; cat
security_soft.txt
--- security.txt ---
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Modified via hard link
--- security_soft.txt ---
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Modified via hard link
```

A change made through the hard link appears through the original name and the symbolic link, since all three paths lead to one inode.

Part E: Rename experiment

```
$ mv security.txt incident.txt
ls -li
total 8
2801705 -rw-r--r-- 2 root root 111 Nov  4 04:09 incident.txt
2801705 -rw-r--r-- 2 root root 111 Nov  4 04:09 security_hard.txt
2801706 lrwxrwxrwx 1 root root  12 Nov  4 04:09 security_soft.txt -> security.txt
```

```
$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Modified via hard link
```

```
$ cat security_soft.txt
cat: security_soft.txt: No such file or directory
```

```
$ readlink security_soft.txt
security.txt
```

Hard link: still works. Renaming only changes one directory entry; the inode, and the other name pointing to it, are untouched. **Symbolic link:** broken. It still stores the text `security.txt`, but no entry with that name exists any more, so the lookup fails with *No such file or directory*. A symbolic link refers to a *name*; a hard link refers to the *inode*.

Part F: Deletion experiment

The links are recreated from scratch so the state is clear.

```
$ rm -f incident.txt security_hard.txt security_soft.txt     # clean up the previous
experiment
printf 'Linux File System Security Lab\nConfidential Security Report\n' > security.txt
ln security.txt security_hard.txt
ln -s security.txt security_soft.txt
ls -li
total 8
2801705 -rw-r--r-- 2 root root 60 Nov  4 04:09 security.txt
2801705 -rw-r--r-- 2 root root 60 Nov  4 04:09 security_hard.txt
2801706 lrwxrwxrwx 1 root root 12 Nov  4 04:09 security_soft.txt -> security.txt
```

```
$ rm security.txt
ls -li
total 4
2801705 -rw-r--r-- 1 root root 60 Nov  4 04:09 security_hard.txt
2801706 lrwxrwxrwx 1 root root 12 Nov  4 04:09 security_soft.txt -> security.txt
```

```
$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
```

```
$ cat security_soft.txt
cat: security_soft.txt: No such file or directory
```

```
$ ls -l security_soft.txt; readlink security_soft.txt; readlink -f security_soft.txt
lrwxrwxrwx 1 root root 12 Nov  4 04:09 security_soft.txt -> security.txt
security.txt
/tmp/lab4/link_lab/security.txt
```

```
$ stat -c 'inode=%i links=%h name=%n' security_hard.txt
inode=2801705 links=1 name=security_hard.txt
```

Dangling (broken) symbolic link: a symlink whose stored path no longer points to an existing file. `ls -l` still lists it (and many terminals colour it red), and `readlink` still prints the stored path, but opening it fails. `readlink -f` prints the path where the target *would* be.

Does deleting the original name delete the data? No. A file's data is freed only when its link count reaches zero (and no process has it open). Before `rm` the count was 2; afterwards `stat` shows the same inode with a link count of **1**, and `cat security_hard.txt` still prints the content. `rm` removes a *name*, not the inode.

Part G: Directory and filesystem behaviour

```
$ mkdir evidence
ln evidence evidence_hard; echo "exit status: $?"
ln: evidence: hard link not allowed for directory
exit status: 1
```

```
$ ln -s evidence evidence_link; echo "exit status: $?"
ls -ld evidence evidence_link
exit status: 0
drwxr-xr-x 2 root root 4096 Nov  4 04:09 evidence
lrwxrwxrwx 1 root root    8 Nov  4 04:09 evidence_link -> evidence
```

A hard link to a directory is **refused** (*hard link not allowed for directory*); a symbolic link to a directory **succeeds**. Hard links to directories could create loops in the directory tree and confuse tools such as `find`, `fsck` and recursive deletes, so the kernel forbids them. Symbolic links can be recognised as links and handled safely.

Cross-filesystem links. The lab machine has two filesystems: the root filesystem and a `tmpfs` mounted on `/dev/shm`.

```
$ df -T . /dev/shm
Filesystem     Type  1K-blocks    Used Available Use% Mounted on
/dev/vda       ext4  264212084 8990816  10441720  47% /
tmpfs          tmpfs   4091692       4   4091688   1% /dev/shm
```

```
$ ln security_hard.txt /dev/shm/xfs_hard.txt; echo "exit status: $?"
ln: failed to create hard link '/dev/shm/xfs_hard.txt' => 'security_hard.txt': Invalid cross-
```

```
device link
exit status: 1
```

```
$ ln -s "$PWD/security_hard.txt" /dev/shm/xfs_soft.txt; echo "exit status: $?"
ls -l /dev/shm/xfs_soft.txt; cat /dev/shm/xfs_soft.txt
exit status: 0
lrwxrwxrwx 1 root root 36 Nov  4 04:09 /dev/shm/xfs_soft.txt -> /tmp/lab4/link_lab/
security_hard.txt
Linux File System Security Lab
Confidential Security Report
```

```
$ rm -f /dev/shm/xfs_soft.txt   # clean up
```

The hard link fails with *Invalid cross-device link* because an inode number is only meaningful inside its own filesystem. The symbolic link works because it stores only a pathname, which the kernel resolves at access time wherever the target is mounted.

Final comparison (from the observations above)

Parameter	Hard Link	Symbolic Link
Inode number	Same as the original	Different (its own inode)
File type	Regular file (`-`)	Symbolic link (`l`)
Link count	2 (shared with the original)	1
File size	Same as the original	Length of the stored pathname (12 bytes here)
Stores target pathname?	No; points to the inode	Yes
readlink output	Nothing (exit status 1)	The stored target path
Effect of modifying target	Visible through every name	Visible (it opens the target)
Effect of renaming target	None; still works	Breaks (dangling)
Effect of deleting target	Data survives while link count > 0	Breaks (dangling)
Can become dangling or broken?	No	Yes
Directory linking	Not allowed	Allowed
Cross-filesystem linking	Not allowed (Invalid cross-device link)	Allowed

sort: Sorting security data

Sample data files were created for each task. `alerts.txt` is the data given in the assignment (IP address, username, failed attempts).

```
$ for f in alerts.txt suspicious_ips.txt ports.txt usernames.txt alerts.csv vuln_report.csv;
do echo "--- $f"; cat $f; done
--- alerts.txt
192.168.1.10 admin 5
10.0.0.8 root 12
172.16.0.5 guest 2
10.0.0.20 admin 8
--- suspicious_ips.txt
192.168.1.25
10.0.0.8
172.16.0.5
10.0.0.8
203.0.113.9
192.168.1.25
10.0.0.20
--- ports.txt
443
22
8080
3306
80
21
3389
22
--- usernames.txt
root
Admin
guest
admin
Alice
bob
Bob
--- alerts.csv
192.168.1.10,admin,5
10.0.0.8,root,12
172.16.0.5,guest,2
10.0.0.20,admin,8
--- vuln_report.csv
ID,Description,Severity
CVE-A,Weak SSH password policy,6.5
CVE-B,Remote code execution in web app,9.8
CVE-C,Outdated TLS configuration,5.3
CVE-D,SQL injection in login form,8.6
CVE-E,Verbose server banner,2.1
```

```
$ head -5 failed_login.log      # full file has 24 entries
--- failed_login.log (24 lines, 22 failed and 2 accepted)
Failed password for invalid user test from 10.0.0.20 port 40019 ssh2
Failed password for invalid user test from 192.168.1.25 port 40009 ssh2
Failed password for invalid user test from 192.168.1.10 port 40023 ssh2
Failed password for invalid user test from 203.0.113.9 port 40016 ssh2
Failed password for invalid user test from 192.168.1.25 port 40011 ssh2
...
```

1. Sort alerts.txt by failed attempts (field 3, numeric, ascending)

```
$ sort -k3,3 -n alerts.txt
172.16.0.5 guest 2
192.168.1.10 admin 5
10.0.0.20 admin 8
10.0.0.8 root 12
```

`-k3,3` sorts on field 3 only, and `-n` compares numbers, so 12 comes after 8 (a text sort would put 12 before 2). Add `-r` for highest first.

1b. Same, highest first

```
$ sort -k3,3 -nr alerts.txt
10.0.0.8 root 12
10.0.0.20 admin 8
192.168.1.10 admin 5
172.16.0.5 guest 2
```

2. Sort suspicious IP addresses alphabetically

```
$ sort suspicious_ips.txt
10.0.0.20
10.0.0.8
10.0.0.8
172.16.0.5
192.168.1.25
192.168.1.25
203.0.113.9
```

This is a plain text sort; it is not numeric order of the octets.

3. Display only unique suspicious IP addresses

```
$ sort -u suspicious_ips.txt
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.25
203.0.113.9
```

4. Sort detected port numbers numerically

```
$ sort -n ports.txt
21
22
22
80
443
3306
3389
8080
```

5. Port numbers in descending numerical order

```
$ sort -nr ports.txt
8080
3389
3306
443
80
22
22
21
```

6. Sort usernames ignoring uppercase/lowercase

```
$ sort -f usernames.txt
Admin
admin
Alice
Bob
```

```
bob
guest
root
```

`-f` folds lower case to upper case for comparison, so `admin` and `Admin` sort together.

7. Count how often each IP occurs in the failed-login log, most frequent first

```
$ grep 'Failed password' failed_login.log | grep -oE '[0-9]+(\.[0-9]+){3}' | sort | uniq -c |
sort -nr
      7 10.0.0.8
      5 192.168.1.25
      4 172.16.0.5
      3 203.0.113.9
      2 198.51.100.7
      2 10.0.0.20
      1 192.168.1.10
```

`uniq -c` only counts *adjacent* duplicates, so the first `sort` is required to bring equal IPs together; the final `sort -nr` orders by the count.

8. Sort IP,username,failed_attempts records by failed attempts

```
$ sort -t, -k3,3 -n alerts.csv
172.16.0.5,guest,2
192.168.1.10,admin,5
10.0.0.20,admin,8
10.0.0.8,root,12
```

`-t,` sets the comma as the field separator.

9. Sort the CSV vulnerability report by severity score (header kept on top, highest score first)

```
$ { head -n 1 vuln_report.csv; tail -n +2 vuln_report.csv | sort -t, -k3,3 -nr; }
ID,Description,Severity
CVE-B,Remote code execution in web app,9.8
CVE-D,SQL injection in login form,8.6
CVE-A,Weak SSH password policy,6.5
CVE-C,Outdated TLS configuration,5.3
CVE-E,Verbose server banner,2.1
```

The header line is printed first so it is not sorted into the data. `-n` handles the decimal scores such as 9.8.

10. Extract IP addresses with grep, then identify unique IPs with sort and uniq

```
$ grep -oE '[0-9]+(\.[0-9]+){3}' failed_login.log | sort | uniq
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
192.168.1.25
192.168.1.30
192.168.1.40
198.51.100.7
203.0.113.9
```

`grep -o` prints only the matched part. `sort -u` gives the same result as `sort | uniq`.

11. Pipeline: top 5 IP addresses by failed login attempts

```
$ grep 'Failed password' failed_login.log | grep -oE 'from [0-9.]+' | awk '{print $2}' | sort
| uniq -c | sort -nr | head -n 5
      7 10.0.0.8
      5 192.168.1.25
      4 172.16.0.5
      3 203.0.113.9
      2 198.51.100.7
```

The stages: select failed logins, extract the source IP, group equal IPs, count them, order by count and keep the first five. The 'Accepted password' lines are excluded by the first `grep`.